

Propuesta de System Prompt v2.0

y rediseño de arquitectura del agente socrático

Cada cambio justificado con evidencia de los logs, la normativa/literatura del diseño original y las restricciones de la arquitectura actual

Autor: Hansell Solís Ramírez · Documento complementario al Informe Integrado · Junio 2026

1. Por qué una v2.0: el diagnóstico que la motiva

Esta propuesta no parte de cero ni de preferencias de estilo: cada cambio responde a una evidencia concreta del Informe Integrado, a una brecha entre lo que el diseño original documentó y lo que la arquitectura efectivamente implementó, o a un requisito que los propios usuarios formularon en el focus group. El cuadro siguiente resume los problemas observados y el cambio que cada uno motiva; las secciones posteriores los desarrollan.

Problema observado	Evidencia (origen)	Cambio propuesto en v2.0
El agente evalúa de más ("excelente", "¡exacto!")	146 de 374 respuestas violan R5 (Informe §7.2-7.3)	Reforzar R5 con prohibición explícita de léxico evaluativo + barrera de post-proceso
Fuga de código bajo atasco	14 respuestas con código que resuelve; coinciden con las copias de P2 y P7 (§7.3, Tabla 5)	Implementar la 3.ª barrera (limpieza de código) que el diseño preveía pero la arquitectura no implementó
Demasiado rígido / poco útil si no afloja	31 señales de atasco; el agente a menudo se mantuvo en preguntas abstractas (§5.3)	Detección de nivel de andamiaje + permiso explícito de "aflojar" a sintaxis vía ejemplo análogo
Inicios improductivos ante enunciado pegado	27 prompts con enunciado en 14 de 20 (§5.3, §6)	Barrera de suficiencia que activa descomposición guiada en vez de "¿qué intentaste?" genérico
Rigidez ante cambio de ejercicio	Casos en que el agente insistió en volver al ejercicio previo (logs P14, P18)	Regla nueva de cambio de ejercicio con reinicio de contexto
No recupera vocabulario técnico	Focus group: distinción declarativo/razonamiento (consenso 3/5)	Modo declarativo: responde directo nombres de funciones/sintaxis
Pregunta lo que el usuario ya sabe	Focus group Insight 5; "que no pregunte lo básico que ya sé"	Adaptación al nivel (modelado del usuario)

Tabla 1. Trazabilidad problema → evidencia → cambio. Ningún cambio se introduce sin un origen verificable.

2. El hallazgo de fondo: tres barreras diseñadas, una implementada

El documento original del system prompt describe, citando a CodeHelp (Liffiton et al., 2023), un pipeline de tres barreras en serie. Sin embargo, ARQUITECTURA.md muestra que la implementación real solo aplica la barrera central (inyectar el prompt principal como system_instruction de Gemini). Las otras dos —y la detección de nivel— quedaron documentadas pero no construidas. Esto explica directamente las desviaciones medidas en el informe.

Barrera diseñada	¿Implementada?	Consecuencia medida en los logs
B1 · Verificación de suficiencia (pre)	No	Inicios genéricos; el enunciado pegado no dispara descomposición
B2 · Respuesta socrática principal	Si	El núcleo socrático funcionó (pocas peticiones directas, R7/R8 cumplidas)
B3 · Limpieza de código (post)	No	14 fugas de código de R1 sin red que las interceptara → copias de P2 y P7
Detección de nivel de andamiaje	No ("el código no controla ni detecta el nivel")	R3 no verificable; el agente no "aflojó" de forma controlada ante atasco

Tabla 2. Brecha diseño-implementación. Origen: system_prompt_agente.pdf (secciones 3-4) contrastado con ARQUITECTURA.md. Es la causa raíz del informe §7.4.

Conclusión de diseño. La v2.0 no “arregla el enfoque socrático” —que funcionó— sino que completa lo que faltaba implementar y corrige el único punto donde el prompt textual no contuvo el sesgo del modelo (R5). Es una mejora de ingeniería más que de filosofía pedagógica.

3. Cambios al system prompt (texto)

Se conserva la estructura y las ocho reglas del prompt original; los cambios son adiciones y refuerzos puntuales, cada uno con su justificación. A continuación, los bloques nuevos o modificados, listos para integrar.

3.1 Refuerzo de R5 (no evaluar) — el cambio más urgente

Razón. R5 fue la regla más violada (146/374, §7 del informe). La causa es que el modelo base tiende a un registro validante; la formulación original (“no digas bien hecho ni incorrecto”) no basta porque no enumera el léxico que el modelo usa por defecto (“excelente”, “¡exacto!”, “perfecto”, “vas por buen camino”).

Justificación normativa. Marco UCR principios 4 y 8 (no sustituir el juicio del estudiante); el diseño original lo vincula a D8.

R5. NO EVALÚES (REFORZADA)
Está PROHIBIDO el léxico evaluativo, incluso como muletilla de apertura. No uses: "excelente", "muy bien", "perfecto", "¡exacto!", "correcto", "vas por buen camino", "buen trabajo". Tampoco confirmes corrección ("así es", "esa es la respuesta"). En su lugar, REDIRIGE con una pregunta de verificación:
- "¿Qué pasa si lo probás con la entrada X?"
- "¿Cómo comprobarías vos mismo si eso es correcto?"
Si necesitás acusar recibo, usá un conector neutro: "Entiendo", "Veamos eso", "Sigamos" – nunca un juicio de valor.

3.2 Refuerzo de R1 (no dar código) con frontera explícita de sintaxis

Razón. Las 14 fugas (§7.3) no fueron soluciones completas, sino fragmentos de sintaxis del propio ejercicio (list(range(n+1)), int(input(...))). El prompt original prohíbe “código completo”, pero no aclara qué pasa con fragmentos parciales del ejercicio asignado.

Justificación. Marco UCR principio 5 (integridad); paper precursor UCR (66.8% teme plagio). La frontera la resuelve R3 vía ejemplo análogo, no con el problema asignado.

R1. PROHIBICIÓN DE SOLUCIÓN DIRECTA (PRECISADA)
No produzcas NINGÚN fragmento de código que pertenezca a la solución del ejercicio ASIGNADO, ni siquiera una línea, aunque sea "solo la sintaxis". Si el estudiante necesita ver sintaxis, muéstrala SIEMPRE sobre un problema ANÁLOGO Y DISTINTO (ver R3 nivel 3): p. ej., si el ejercicio rota una lista de números, ilustrá .pop().append() con una lista de frutas, nunca con la lista del ejercicio.

3.3 R3 reescrita: andamiaje por niveles con permiso de “aflojar”

Razón. Este es el punto que motivó tu pedido: “el prompt no servía de mucho si no daba al menos sintaxis con ejemplos”. Los logs muestran 31 señales de atasco; el agente con frecuencia se mantuvo en preguntas abstractas sin escalar, lo que produce la presión temporal del NASA-TLX y el tedio del SUS. La solución no es abandonar lo socrático, sino hacer el escalamiento OBLIGATORIO y observable.

Justificación. Vygotsky (ZDP) y Wood et al. (1976): el andamiaje debe ajustarse al nivel; si la ayuda no destraba, debe volverse más concreta. Alshaikh (2020) y D3 del diseño original ya lo prevén; la v2.0 lo hace ejecutable.

R3. ANDAMIAJE GRADUAL OBLIGATORIO (3 NIVELES)
Mantén un contador interno de "intentos sin destrabar" para el subproblema actual. Decláralo al inicio de tu respuesta en una etiqueta oculta para el log: [NIVEL=n].
Nivel 1: pregunta amplia de descomposición.
Nivel 2 (tras 1 señal de atasco o respuesta vaga): pista conceptual sobre QUÉ estructura aplicar.
Nivel 3 (tras 2 señales de atasco): ejemplo de sintaxis con un problema ANÁLOGO y DISTINTO. Aquí SÍ mostrás código, pero del análogo, jamás del ejercicio asignado.
REGLA DE AFLOJE: si tras el Nivel 3 el estudiante sigue atascado, no repitas preguntas; descomponé el siguiente paso mínimo en una microacción concreta ("escribí el encabezado de la función y probala vacía"). Nunca dejes al estudiante sin una acción posible.

3.4 Reglas nuevas: cambio de ejercicio y cierre de atasco

Razón (R9). En los logs, varios estudiantes intentaron pasar al siguiente ejercicio y el agente insistió en volver al anterior (p. ej., P14, P18), lo que generó fricción innecesaria; la actividad tenía 5 ejercicios independientes.

R9. CAMBIO DE EJERCICIO PERMITIDO
Si el estudiante presenta un enunciado nuevo o dice "pasemos al siguiente", aceptá el cambio, reiniciá el contador de nivel y no lo obligues a terminar el anterior. La actividad tiene 5 ejercicios independientes.

Razón (R10). P3 fue sensibilidad a la frustración: el patrón P3 del diseño original existe, pero al no haber detección de nivel no se disparaba de forma fiable. Se ata ahora al contador de R3.

R10. SENSIBILIDAD A LA FRUSTRACIÓN (ATADA A NIVEL)
Si el contador de atasco llega a 2 Y aparece lenguaje emocional negativo, reconocé la emoción, BAJÁ un nivel de exigencia y ofrecé la microacción de R3, no otra pregunta abierta.

3.5 Regla nueva R11: modo declarativo (validada por usuarios)

Razón. Es el aporte directo del focus group. Los participantes distinguen con claridad entre razonamiento (dominio del método socrático) y vocabulario declarativo: el nombre de una función, una sintaxis exacta. Hacer preguntas socráticas para que alguien "adivine" que el método se llama pop() no enseña, irrita: "yo nunca voy a adivinar el nombre de la función". La respuesta directa a vocabulario tuvo consenso 3/5 como característica innegociable.

Justificación. Distingue integridad (no dar la solución del ejercicio, que se mantiene) de eficiencia (no negar un dato declarativo que el estudiante ya sabría buscar en la documentación). Alinea el agente con el dominio donde el método socrático es legítimo.

R11. MODO DECLARATIVO PARA VOCABULARIO TÉCNICO
Distinguí dos tipos de consulta:
(a) RAZONAMIENTO (cómo resolver, qué lógica, por qué falla): aplicá el método socrático normal (R2, R3).
(b) DECLARATIVA (el NOMBRE de un método, la firma de una función, una sintaxis puntual que no resuelve el ejercicio): respondé DIRECTO y breve, y devolvé el control al estudiante.
Ejemplo: "¿cómo se llama el método que saca el último elemento?"
-> "Es pop(). ¿Cómo lo usarías en tu caso?" (das el dato, no la solución).
REGLA DE BORDE: si dar el dato declarativo equivale a entregar la solución del ejercicio asignado, NO aplica R11; volvé a R1.

3.6 Techo de fricción configurable por el usuario

Razón. El focus group fijó un umbral de tolerancia de 3 a 6 intercambios antes de que la guía socrática se vuelva contraproducente, con consenso alto (4/5) sobre tener un “límite configurable de preguntas antes de respuesta directa”: “pregúntenme 3 y ya después denme la respuesta”.

Justificación. Combina la regla de afloje (R3) con control del usuario: el estudiante fija su propio techo. No abandona lo socrático; lo acota a la zona donde el propio usuario lo percibe productivo.

```
TECHO CONFIGURABLE (parámetro de sesión)
El estudiante puede fijar N (por defecto 4, rango 3-6). Cuando el
contador de intercambios sin destrabar alcanza N, el agente pasa
a una respuesta más directa (nivel 3 + microacción), reconociendo
el límite: “Llevamos N intentos; te doy un empujón más concreto.”
```

4. Rediseño de arquitectura: implementar las 3 barreras + detección de nivel

El prompt v2.0 solo funciona si la arquitectura ejecuta lo que el texto promete. La arquitectura actual hace una sola llamada a Gemini con system_instruction. La propuesta mantiene ese núcleo y añade dos llamadas envolventes y un registro de nivel, siguiendo el pipeline de tres etapas de CodeHelp que el diseño ya citaba pero no implementó.

4.1 Flujo actual vs. propuesto

```
ACTUAL (1 etapa):
  POST /api/chat
    -> Gemini(system_instruction + history + msg) -> respuesta

PROPUESTO (3 barreras + nivel):
  POST /api/chat
    1) B1 Suficiencia: Gemini(prompt_suficiencia, msg)
      -> {suficiente:bool, faltante:str}
      si false -> responder pidiendo lo faltante (sin gastar B2)
    2) B2 Principal: Gemini(system_instruction_v2 + history + msg)
      -> respuesta + etiqueta [NIVEL=n]
    3) B3 Limpieza: Gemini(prompt_limpieza, respuesta)
      -> respuesta sin código del ejercicio asignado
    4) extraer [NIVEL=n] -> guardar en log -> quitar del texto
      -> devolver respuesta limpia
```

Figura/esquema 1. El núcleo (B2) no cambia; se le envuelve con las dos barreras que faltaban y se persiste el nivel. Origen del diseño: Liffiton et al. (2023), citado en el PDF original §4.

4.2 Barrera 1 — Verificación de suficiencia (resuelve el enunciado pegado)

Razón. 27 prompts con enunciado pegado en 14 participantes. Sin B1, el agente responde con un “¿qué intentaste?” genérico que no orienta. B1 detecta el caso y activa descomposición.

Implementación. Endpoint /api/chat ejecuta primero el prompt de suficiencia (ya redactado en el PDF original §4.1). Si suficiente=false por enunciado pegado, el plugin responde con una plantilla fija de descomposición (entrada, salida, restricción, primer paso), sin invocar B2 — ahorra latencia y enseña a formular.

```
if (esEnunciadoPegado(msg) || !suficiencia.suficiente) {
  return plantillaDescomposicion(faltante); // no llama a B2
}
```

4.3 Barrera 3 — Limpieza de código (resuelve las fugas de R1)

Razón. Las 14 fugas y las 4 copias de código (P2, P7) habrían sido interceptadas por esta barrera. Es la red de seguridad post-respuesta que el diseño preveía.

Implementación. Tras B2, se pasa la respuesta por el prompt de limpieza (PDF original §4.2), que reescribe cualquier bloque de código del ejercicio explicándolo con palabras. Costo: una llamada extra por turno; beneficio: cierra el agujero que produjo la conducta de copia.

4.4 Detección de nivel — el mecanismo para “aflojar” de forma observable

Razón. R3 era no verificable porque “el código no controla ni detecta el nivel”. Sin un registro de nivel no se puede saber si el agente escaló ni evaluar el andamiaje en la tesis. La v2.0 lo vuelve un dato medible.

Implementación de bajo costo. No requiere un clasificador nuevo: el prompt v2 (R3) pide al modelo emitir una etiqueta [NIVEL=n] al inicio de cada respuesta. El servidor la extrae con una expresión regular, la guarda como nuevo tipo de evento en el log y la elimina del texto antes de mostrarla. Así el nivel queda persistido por turno y se puede graficar la trayectoria de andamiaje de cada sesión.

```
const m = respuesta.match(/^\[NIVEL=(\d+)\]/);
const nivel = m ? Number(m[1]) : null;
respuesta = respuesta.replace(/^\[NIVEL=\d+\]\s*/, '');
log(sessionId, 'SCAFFOLD_LEVEL', { nivel }); // nuevo evento
```

Beneficio para la tesis. Con SCAFFOLD_LEVEL en los logs, el próximo análisis podrá responder empíricamente “¿el agente escaló cuando el estudiante se atascó?” —algo que hoy es imposible— y medir si “aflojar” reduce la frustración del NASA-TLX.

4.5 Mitigaciones de robustez (confusor técnico del informe)

Razón. 37 fallas de servicio (35 Gemini 503) afectaron a 17 de 20 y contaminaron la frustración percibida (Informe §5.4).

- **Reintento con backoff:** ante un 503, reintentar 2-3 veces con espera creciente antes de mostrar error, para que la falla sea invisible al estudiante en la mayoría de los casos.
- **Mensaje de estado claro:** si el reintento falla, mostrar “el servicio está ocupado, reintentando...” en vez de un error crudo, y registrar el evento para separarlo de la fricción pedagógica.
- **Registrar versión del prompt:** persistir la versión (v2.0.0) en cada log, como pide el principio 2 del Marco UCR (rendición de cuentas), para trazabilidad entre experimentos.

5. Resumen de cambios y su trazabilidad

Cambio v2.0	Tipo	Evidencia que lo justifica
R5 reforzada con léxico prohibido	Prompt	146 violaciones de R5 (Informe §7)
R1 con frontera de sintaxis	Prompt	14 fugas de código = copias de P2/P7 (§7.3)
R3 con niveles obligatorios y afloje	Prompt	31 atascos; presión temporal NASA (§2,§5)
R9 cambio de ejercicio	Prompt	Fricción observada en P14/P18
R10 frustración atada a nivel	Prompt	Frustración NASA 58.5; P3 sin disparador fiable
B1 suficiencia	Arquitectura	27 enunciados pegados (§5.3)
B3 limpieza de código	Arquitectura	Fugas R1 sin red (§7.4)
Detección de nivel (SCAFFOLD_LEVEL)	Arquitectura	R3 no verificable hoy (§7.4)
R11 modo declarativo	Prompt	Focus group: distinción declarativo/razonamiento
Techo de fricción configurable	Prompt/UI	Focus group: umbral 3-6, consenso 4/5
Reintento ante 503	Arquitectura	37 fallas técnicas (§5.4)

Tabla 3. Toda mejora rastrea a una evidencia del informe o a la brecha diseño-implementación. Cumple el principio 2 del Marco UCR (rendición de cuentas).

6. Limitaciones de la propuesta

- Las tres barreras triplican el número de llamadas a Gemini por turno: aumentan costo y latencia. Debe medirse si la latencia adicional reintroduce presión temporal.
- La etiqueta [NIVEL=n] depende de que el modelo la emita con fidelidad; conviene validar en pilotaje que no la omita ni la invente.
- El refuerzo de R5 reduce el léxico evaluativo, pero el sesgo del modelo base puede reaparecer; la barrera de limpieza no cubre R5, por lo que podría añadirse una verificación de tono.
- Estos cambios deben probarse en el pilotaje cerrado (3-5 estudiantes) que el documento original ya prevé, y versionarse como v2.0.0 antes del próximo experimento.